WhatNext Vision Motors

Salesforce Project Walkthrough

Custom objects, automation (Flows) and Apex for a vehicle sales and service business

Team Leader: Rajkumar

Team Members: Siva Govindh, Ramesh Sachin, Aswin

Platform: Salesforce Developer Edition (Lightning Experience)

1. Project Overview

WhatNext Vision Motors is a Salesforce application that manages vehicles, dealers, customers, orders, test drives and service requests. It also automates three business rules:

- **Auto dealer assignment:** every new order is linked to the dealer located in the customer's city.
- **Stock control:** an order cannot be confirmed when the vehicle is out of stock, and each confirmed order reduces stock by one.
- **Reminders and batch processing:** customers get a test drive reminder email one day before, and pending orders are confirmed automatically every night.

Build summary

Milestone	What was built	Tool used
1	Developer org	developer.salesforce.com
2	6 custom objects	Object Manager
3	6 custom tabs	Setup > Tabs
4	Lightning app: WhatNext Vision Motors	App Manager
5	Fields and relationships (lookups, picklists, etc.)	Object Manager
6	2 record-triggered Flows	Flow Builder
7	Apex trigger, handler, batch class and scheduler	Developer Console

2. Milestone 1 - Developer Org

A free Developer Edition org was created at developer.salesforce.com/signup (Role: Developer, Country: India). After verifying the email and setting a password, the Setup home page opens. All work in this project was done in this org.

3. Milestone 2 - Custom Objects

Six custom objects were created in Setup > Object Manager > Create > Custom Object, each with Allow Reports and Allow Search enabled.

Object (label)	Record name field	Purpose
Vehicle	Vehicle Name	Vehicles for sale, with stock and price
Vehicle Dealer	Dealer Name	Dealers and their locations
Vehicle Customer	Customer Name	Customer contact details and address
Vehicle Order	Vehicle Order Name	Orders placed by customers
Vehicle Test Drive	Test Drive Name	Scheduled test drives
Vehicle Service Request	Service Request Name	Service requests for vehicles

4. Milestone 3 - Custom Tabs

A tab was created for each object (Setup > Tabs > Custom Object Tabs > New) with the visibility 'Default On' for all profiles.

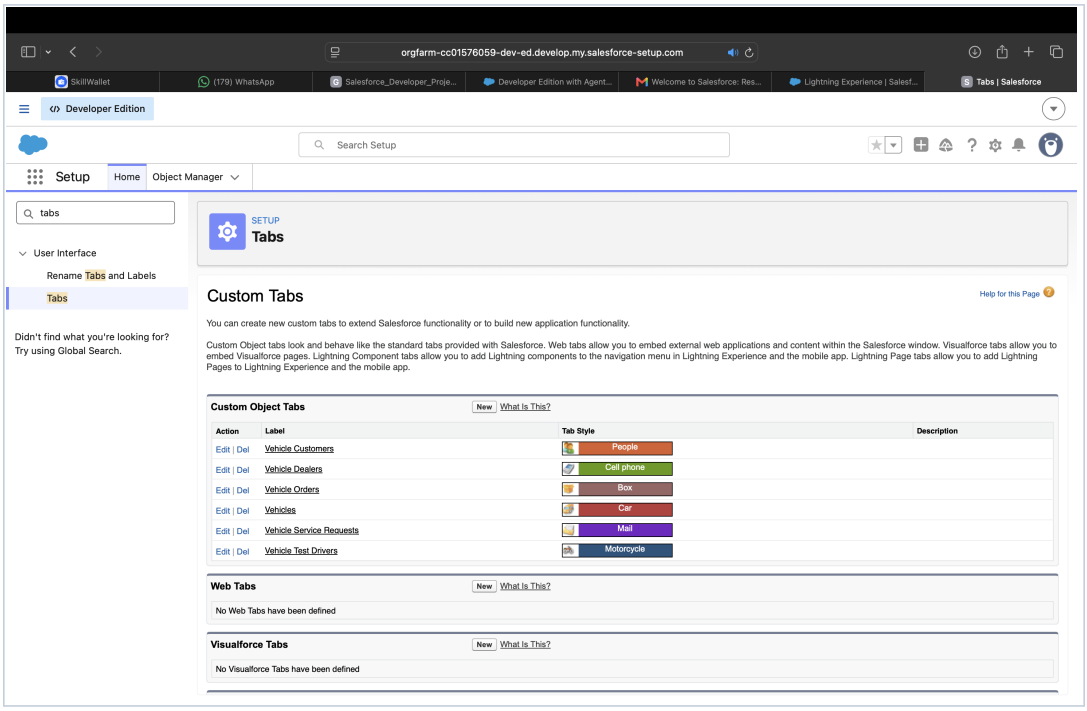


Figure 1: All six custom object tabs created.

5. Milestone 4 - Lightning App

The app was created in Setup > App Manager > New Lightning App with the name **WhatNext Vision Motors**. The navigation items are the six custom objects plus Reports and Dashboards, and the System Administrator profile has access.

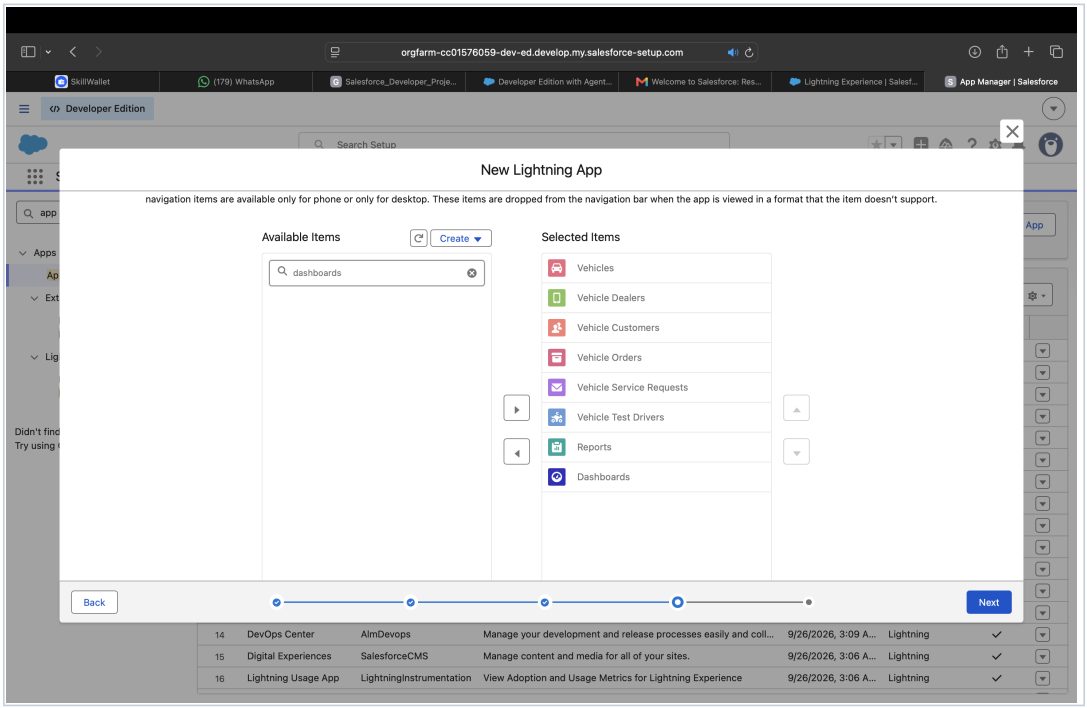


Figure 2: Navigation items selected for the app.

6. Milestone 5 - Fields and Relationships

Fields were added from Object Manager > (object) > Fields & Relationships > New. The field API names matter because the Apex code and Flows use them.

Object	Field label	Type	Details
Vehicle	Vehicle Model	Picklist	Sedan, SUV, EV
Vehicle	Stock Quantity	Number (18,0)	Used by the stock rules
Vehicle	Price	Currency (16,2)	
Vehicle	Status	Picklist	Available, Out of Stock, Discontinued
Vehicle	Dealer	Lookup	Vehicle Dealer
Vehicle Dealer	Dealer Location	Text (255)	City, matched with customer address
Vehicle Dealer	Dealer Code	Auto Number	Format DLR-{0000}
Vehicle Dealer	Phone / Email	Phone / Email	
Vehicle Customer	Email / Phone	Email / Phone	
Vehicle Customer	Address	Text (255)	City name
Vehicle Customer	Preferred Vehicle Type	Picklist	Sedan, SUV, EV
Vehicle Order	Vehicle Customer	Lookup	Vehicle Customer
Vehicle Order	Vehicle	Lookup	Vehicle
Vehicle Order	Dealer	Lookup	Vehicle Dealer (filled by Flow 1)
Vehicle Order	Order Date	Date	
Vehicle Order	Status	Picklist	Pending, Confirmed, Delivered, Canceled
Vehicle Test Drive	Vehicle Customer / Vehicle	Lookups	
Vehicle Test Drive	Test Drive Date	Date	Used by the reminder Flow
Vehicle Test Drive	Status	Picklist	Scheduled, Completed, Canceled
Vehicle Service Request	Vehicle Customer / Vehicle	Lookups	
Vehicle Service Request	Service Date	Date	
Vehicle Service Request	Issue Description	Text (255)	
Vehicle Service Request	Status	Picklist	Requested, In Progress, Completed

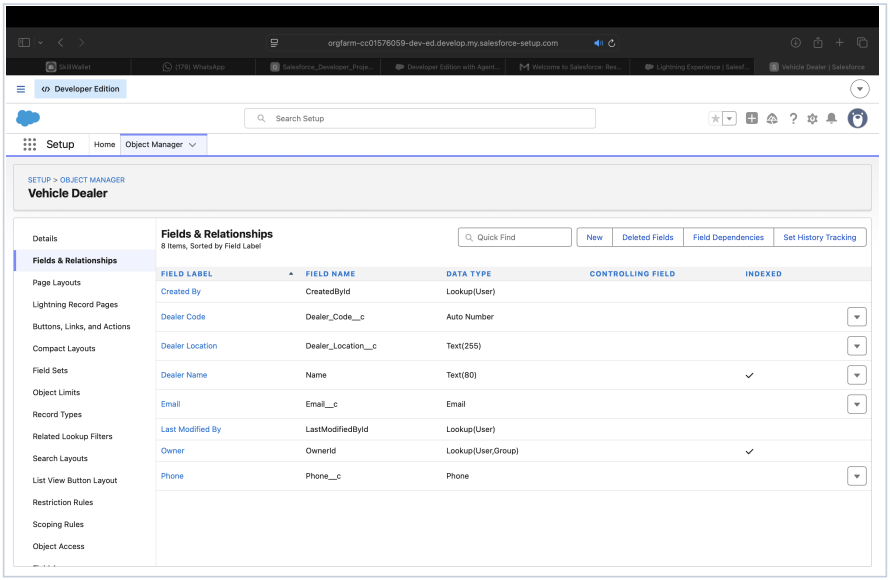


Figure 3: Vehicle Dealer fields (Dealer Code, Dealer Location, Email, Phone).

7. Milestone 6 - Flows

Flow 1: Auto Assign Dealer

Type: Record-Triggered Flow on **Vehicle Order**, triggered when a record is created, optimized for Actions and Related Records.

- **Get Customer Information** (Get Records): Vehicle Customer where Record ID equals the order's Vehicle Customer.
- **Get Nearest Dealer** (Get Records): Vehicle Dealer where Dealer Location equals the customer's Address.
- **Assign Dealer to Order** (Update Records): updates the triggering order, setting Dealer to the dealer's Id.

The match is an exact text match between customer address and dealer location (for example, both 'Chennai'). True distance-based matching would need Geolocation fields.

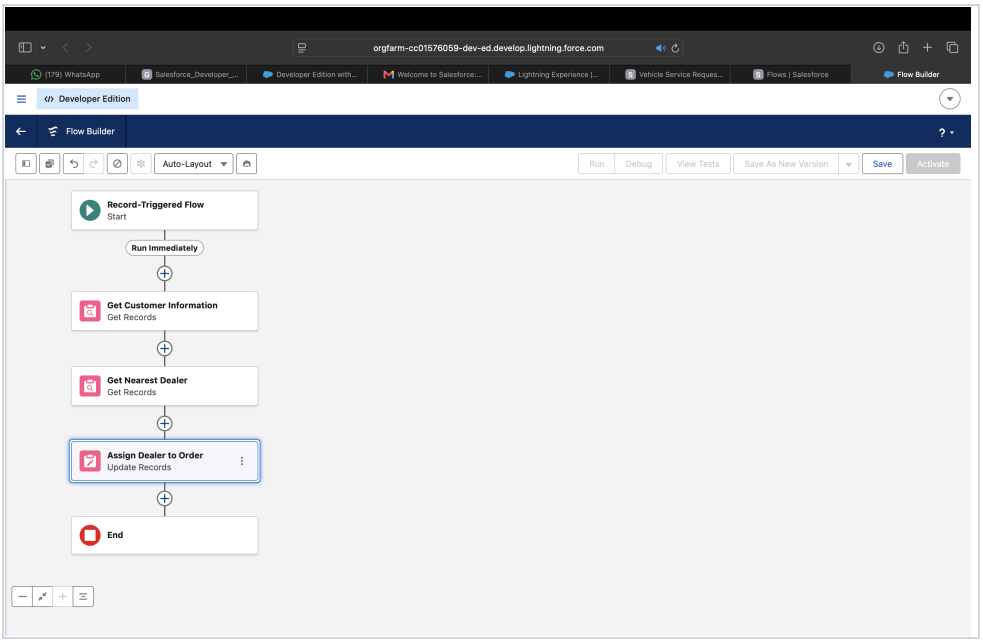


Figure 4: Auto Assign Dealer flow canvas.

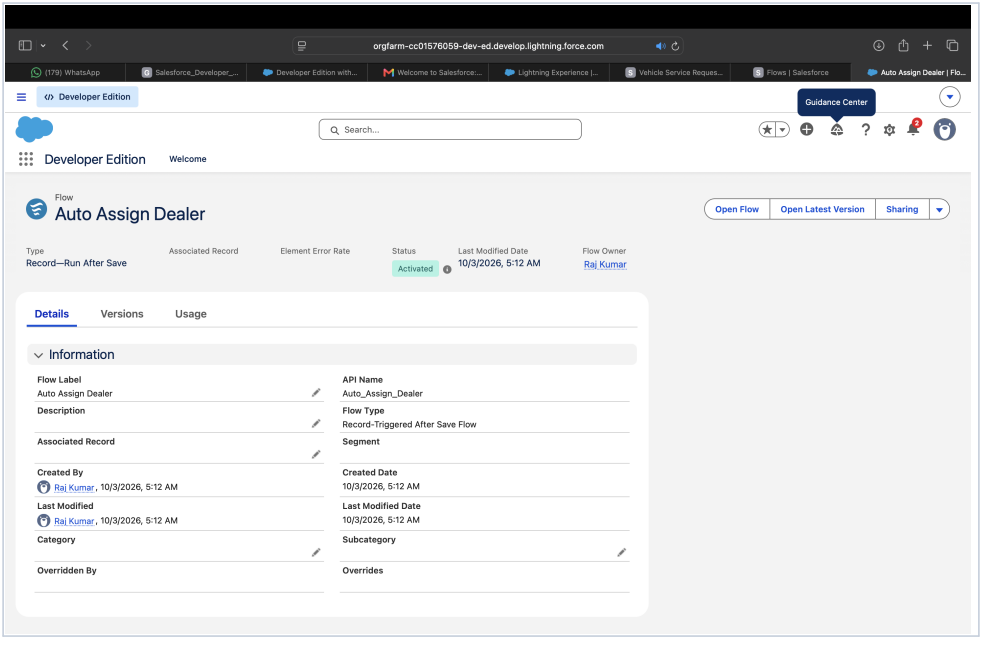


Figure 5: Flow saved and shown as Activated.

Flow 2: Test Drive Reminder

Type: Record-Triggered Flow on **Vehicle Test Drive**, created or updated, entry condition Status equals Scheduled.

- **Scheduled path** 'Reminder Before Test Drive': time source Test Drive Date, offset 1 day before.
- **Get Customer Information** (Get Records) on the scheduled path.
- **Send Test Drive Reminder** (Send Email action): sends the customer an email with the test drive date.

The Get Records and Send Email elements must sit on the scheduled path, not on Run Immediately. Otherwise the email would be sent right away instead of one day before.

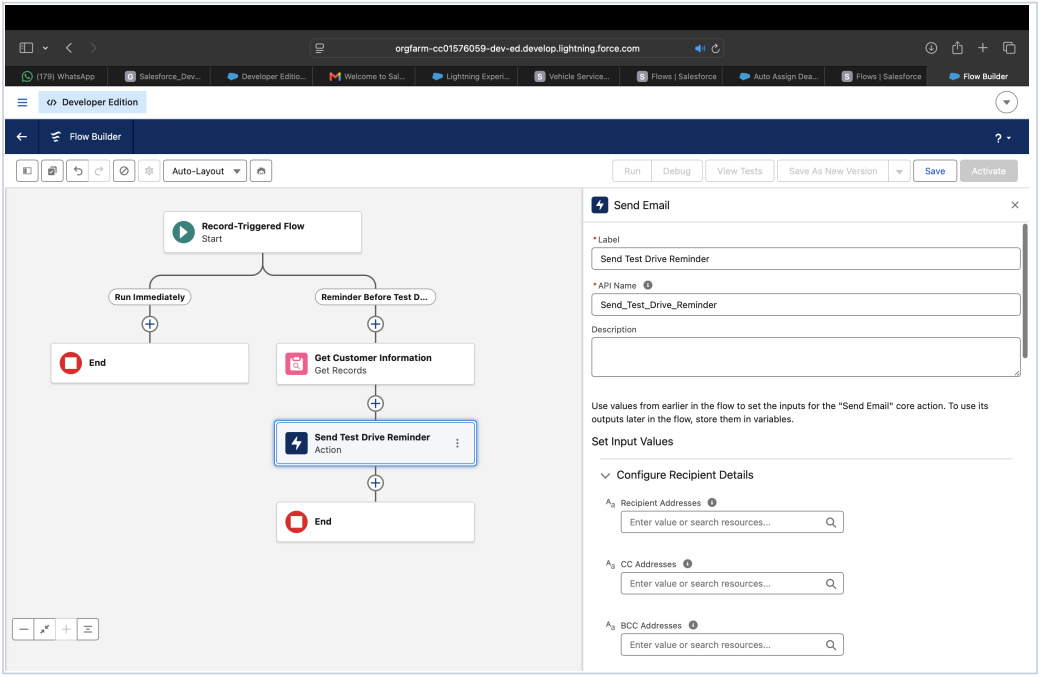


Figure 6: Send Test Drive Reminder action on the scheduled path.

8. Milestone 7 - Apex

Four Apex items were created in the Developer Console (File > New). Full source code is in the appendix.

Item	Type	What it does
VehicleOrderTriggerHandler	Apex class	Holds the logic. Before save: blocks an order if the vehicle stock is 0. After save: reduces stock by 1 for each order that newly becomes Confirmed, and sets the vehicle to Out of Stock at 0.
VehicleOrderTrigger	Apex trigger	Runs on Vehicle Order (before/after insert and update) and calls the handler.
VehicleOrderBatch	Batch class	Finds all Pending orders and confirms them while stock is available.
VehicleOrderBatchScheduler	Schedulable class	Runs the batch (50 records at a time).

Scheduling: the job was scheduled from Execute Anonymous with the cron expression `0 0 0 * * ?` (every day at 12:00 AM).

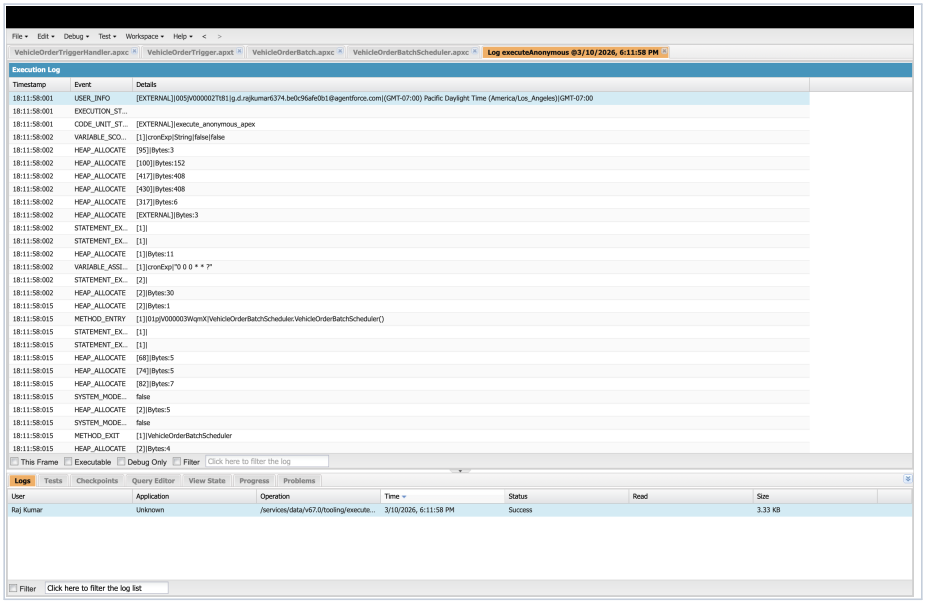


Figure 7: Execute Anonymous log with Status Success (scheduler created).

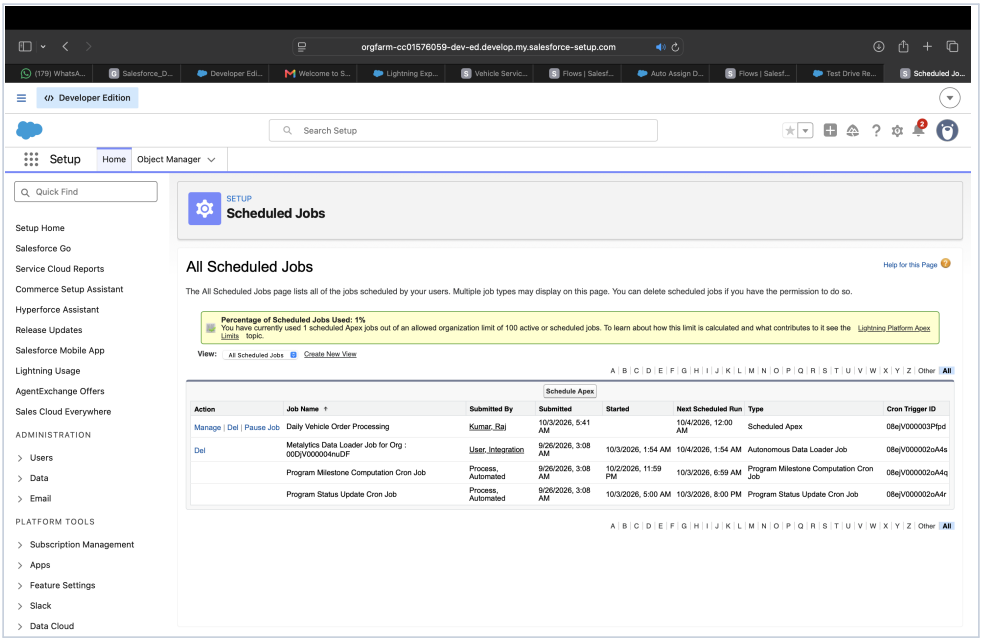


Figure 8: 'Daily Vehicle Order Processing' in Scheduled Jobs, next run 10/4/2026 12:00 AM.

9. Testing and Results

Test data: dealer 'Chennai Motors' (location Chennai), vehicle 'Test SUV', customer 'Test Customer' (address Chennai).

Test	Action	Expected	Result
Auto dealer	Create a Confirmed order with the Dealer empty	Dealer becomes Chennai Motors	Passed
Stock block	Stock = 0, create a Confirmed order	Error: vehicle out of stock, order not saved	Passed
Stock reduction	Stock = 5, create a Confirmed order	Stock becomes 4	Passed
Nightly batch	Create a Pending order, run the batch	Order becomes Confirmed, stock 4 to 3	Passed
Test drive reminder	Create a Scheduled test drive for 10/5/2026	Saves; email sent 10/4/2026	Saved (email due 10/4)

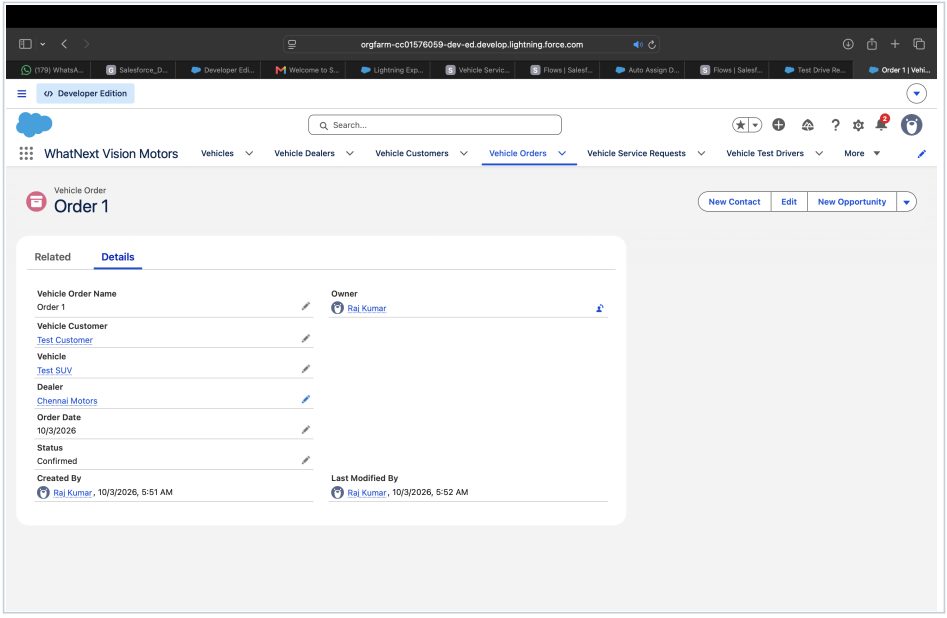


Figure 9: Order with Dealer = Chennai Motors, assigned automatically by Flow 1.

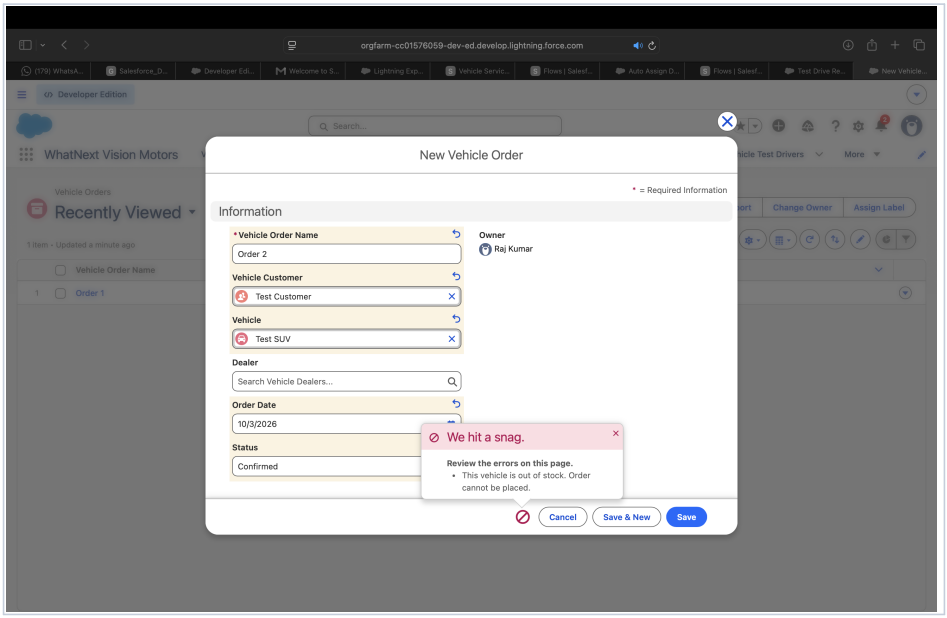


Figure 10: Out-of-stock error from the trigger.

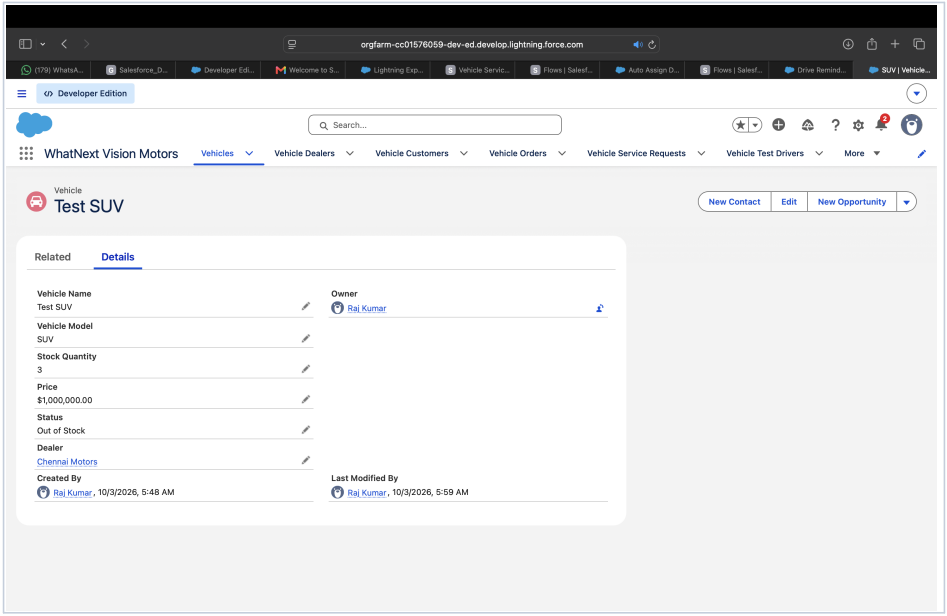


Figure 11: Vehicle stock after the batch confirmed a pending order (stock 3).

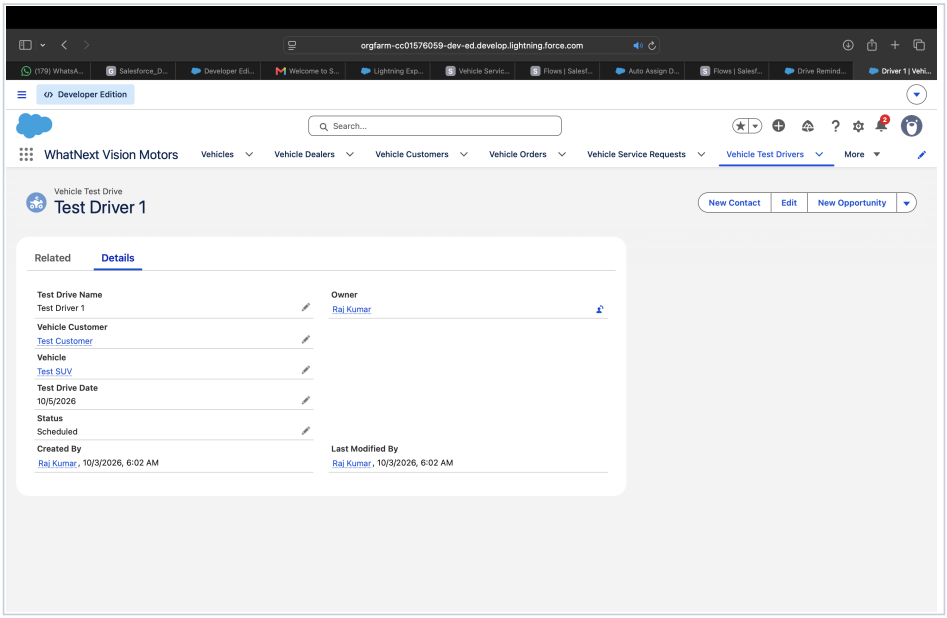


Figure 12: Scheduled test drive saved for 10/5/2026.

10. Design Notes

- Stock is reduced only once per order, when it newly becomes Confirmed, so later edits do not reduce stock again.
- The batch only changes the order status. The trigger handles the stock reduction, so stock is never reduced twice.
- Several orders for the same vehicle in one save are counted correctly.
- The vehicle Status is set to Out of Stock when stock reaches 0 but is not reset automatically when stock is added.
- Flow 1 needs the extra **Dealer** lookup on Vehicle Order, which stores the assigned dealer.

Appendix - Apex Source Code

VehicleOrderTriggerHandler.cls

```
public with sharing class VehicleOrderTriggerHandler {

    public static void handleTrigger(
        List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldMap,
        Boolean isBefore, Boolean isAfter,
        Boolean isInsert, Boolean isUpdate
    ) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders, oldMap, isInsert);
        }
        if (isAfter && (isInsert || isUpdate)) {
            reduceStockForConfirmedOrders(newOrders, oldMap);
        }
    }

    // True only when the order is Confirmed NOW and was not Confirmed before.
    // This stops stock being reduced again on every later edit of the order.
    private static Boolean becomesConfirmed(Vehicle_Order__c ord, Map<Id, Vehicle_Order__c> oldMap) {
        if (ord.Vehicle__c == null || ord.Status__c != 'Confirmed') return false;
        if (oldMap == null) return true;
        Vehicle_Order__c oldOrd = oldMap.get(ord.Id);
        return oldOrd == null || oldOrd.Status__c != 'Confirmed';
    }

    private static Map<Id, Decimal> loadStock(Set<Id> vehicleIds) {
        Map<Id, Decimal> stock = new Map<Id, Decimal>();
        for (Vehicle__c v : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
            stock.put(v.Id, v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c);
        }
        return stock;
    }

    // BEFORE: block orders for out-of-stock vehicles
    private static void preventOrderIfOutOfStock(
        List<Vehicle_Order__c> orders, Map<Id, Vehicle_Order__c> oldMap, Boolean isInsert
    ) {
        Set<Id> vehicleIds = new Set<Id>();
        List<Vehicle_Order__c> toCheck = new List<Vehicle_Order__c>();
        for (Vehicle_Order__c o : orders) {
            if (o.Vehicle__c == null) continue;
            if (isInsert || becomesConfirmed(o, oldMap)) {
                toCheck.add(o);
                vehicleIds.add(o.Vehicle__c);
            }
        }
        if (toCheck.isEmpty()) return;

        Map<Id, Decimal> remaining = loadStock(vehicleIds);
        for (Vehicle_Order__c o : toCheck) {
            Decimal qty = remaining.containsKey(o.Vehicle__c) ? remaining.get(o.Vehicle__c) : 0;
            if (qty <= 0) {
                o.addError('This vehicle is out of stock. Order cannot be placed.');
            } else if (o.Status__c == 'Confirmed') {
                remaining.put(o.Vehicle__c, qty - 1); // handles several orders for the same vehicle in one save
            }
        }
    }

    // AFTER: reduce stock once per newly confirmed order
    private static void reduceStockForConfirmedOrders(
        List<Vehicle_Order__c> orders, Map<Id, Vehicle_Order__c> oldMap
    ) {
        Map<Id, Integer> countByVehicle = new Map<Id, Integer>();
        for (Vehicle_Order__c o : orders) {
            if (becomesConfirmed(o, oldMap)) {
                Integer c = countByVehicle.containsKey(o.Vehicle__c) ? countByVehicle.get(o.Vehicle__c) : 0;
                countByVehicle.put(o.Vehicle__c, c + 1);
            }
        }
        if (countByVehicle.isEmpty()) return;

        List<Vehicle__c> toUpdate = new List<Vehicle__c>();
        for (Vehicle__c v : [SELECT Id, Stock_Quantity__c, Status__c FROM Vehicle__c WHERE Id IN :countByVehicle.keySet()]) {
            Decimal qty = v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c;
            qty = qty - countByVehicle.get(v.Id);
            v.Stock_Quantity__c = qty < 0 ? 0 : qty;
            if (v.Stock_Quantity__c == 0) v.Status__c = 'Out of Stock';
```

```
            toUpdate.add(v);
        }
        update toUpdate;
    }
}
```

VehicleOrderTrigger.trigger

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new, Trigger.oldMap,
        Trigger.isBefore, Trigger.isAfter,
        Trigger.isInsert, Trigger.isUpdate
    );
}
```

VehicleOrderBatch.cls

```
global class VehicleOrderBatch implements Database.Batchable<SObject>, Database.Stateful {

    global Integer confirmedCount = 0;

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending' AND Vehicle__c != null
            ORDER BY CreatedDate ASC
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c o : orderList) vehicleIds.add(o.Vehicle__c);

        Map<Id, Decimal> remaining = new Map<Id, Decimal>();
        for (Vehicle__c v : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
            remaining.put(v.Id, v.Stock_Quantity__c == null ? 0 : v.Stock_Quantity__c);
        }

        List<Vehicle_Order__c> toConfirm = new List<Vehicle_Order__c>();
        for (Vehicle_Order__c o : orderList) {
            Decimal qty = remaining.containsKey(o.Vehicle__c) ? remaining.get(o.Vehicle__c) : 0;
            if (qty > 0) {
                o.Status__c = 'Confirmed';   // stock is reduced by VehicleOrderTrigger, not here
                remaining.put(o.Vehicle__c, qty - 1);
                toConfirm.add(o);
            }
            // else: stays 'Pending'
        }

        if (!toConfirm.isEmpty()) {
            Database.SaveResult[] results = Database.update(toConfirm, false);
            for (Database.SaveResult sr : results) {
                if (sr.isSuccess()) confirmedCount++;
            }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch completed. Orders confirmed: ' + confirmedCount);
    }
}
```

VehicleOrderBatchScheduler.cls

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        Database.executeBatch(new VehicleOrderBatch(), 50); // 50 = batch size
    }
}

// Run once in Developer Console > Debug > Open Execute Anonymous Window:
//
// String cronExp = '0 0 0 * * ?';   // every night at 12:00 AM
// System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Schedule command (Execute Anonymous):

```
String cronExp = '0 0 0 * * ?';
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```